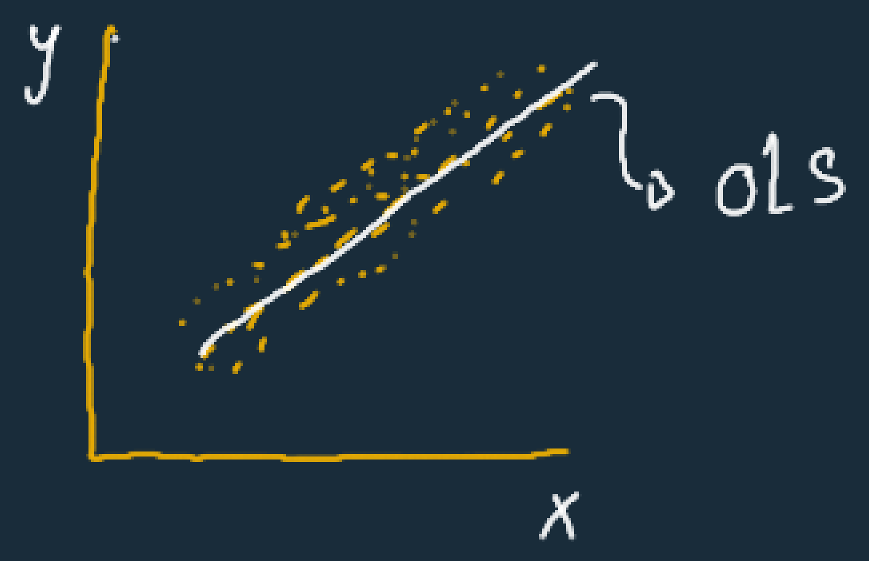
Linear Models

SVM

Linear Regression



$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots \theta_n x_n$$

$$\hat{y} = \theta^T X$$

Logistic Regression

Decision Boundary

$$z = \theta^T X$$

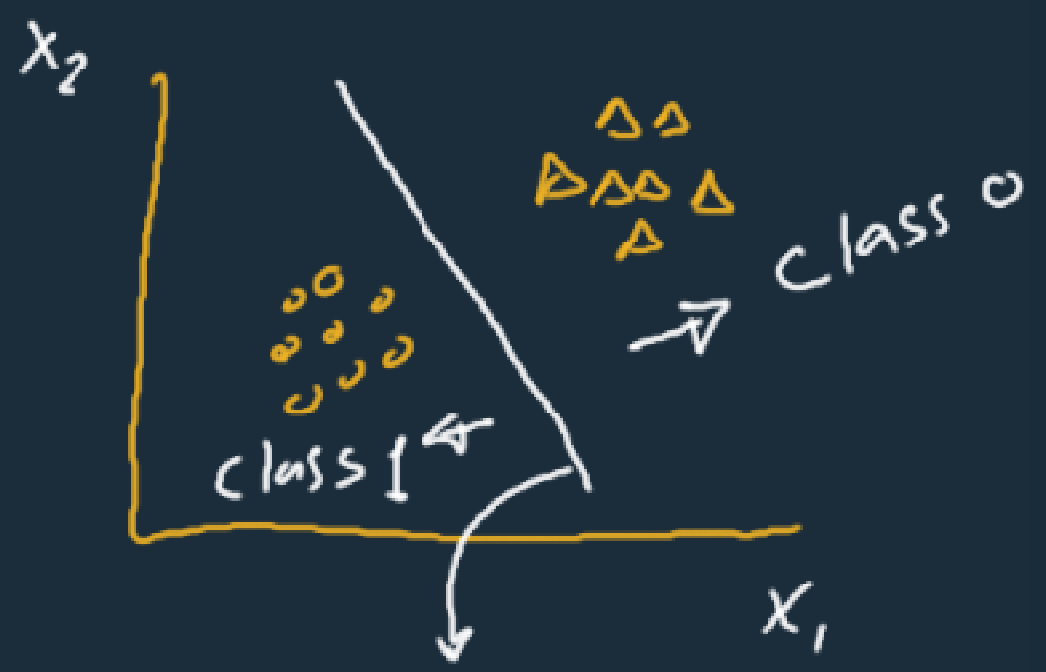
$$\hat{y} = \sigma(z) \rightarrow P(y==1)$$

Linear SVC

* Decision Boundary
* Cost fn

Log loss

Logistic Regression

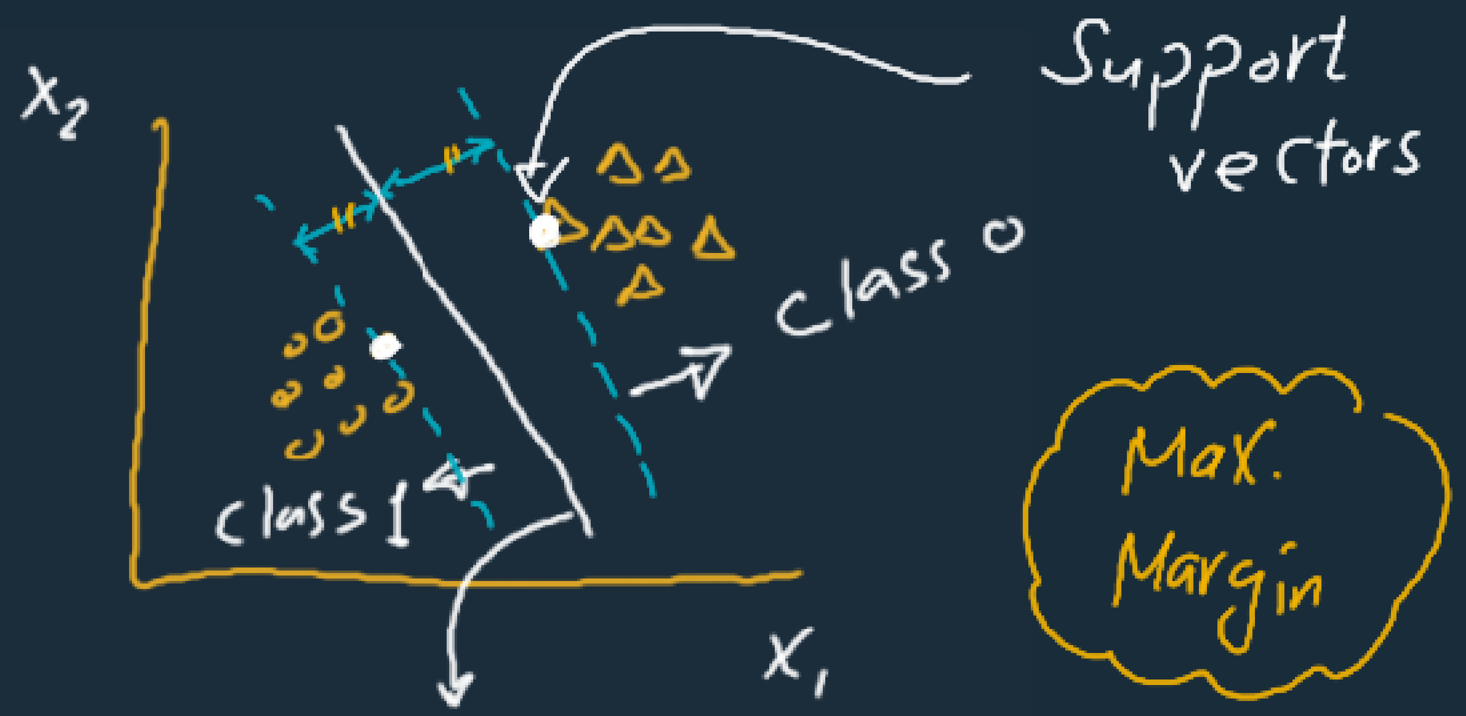


Decision Boundary

$$\theta^T x = 0$$

- class 1 : $\theta^T x \geq 0$ ($P \geq 0.5$)
- class 0 : $\theta^T x < 0$ ($P < 0.5$)

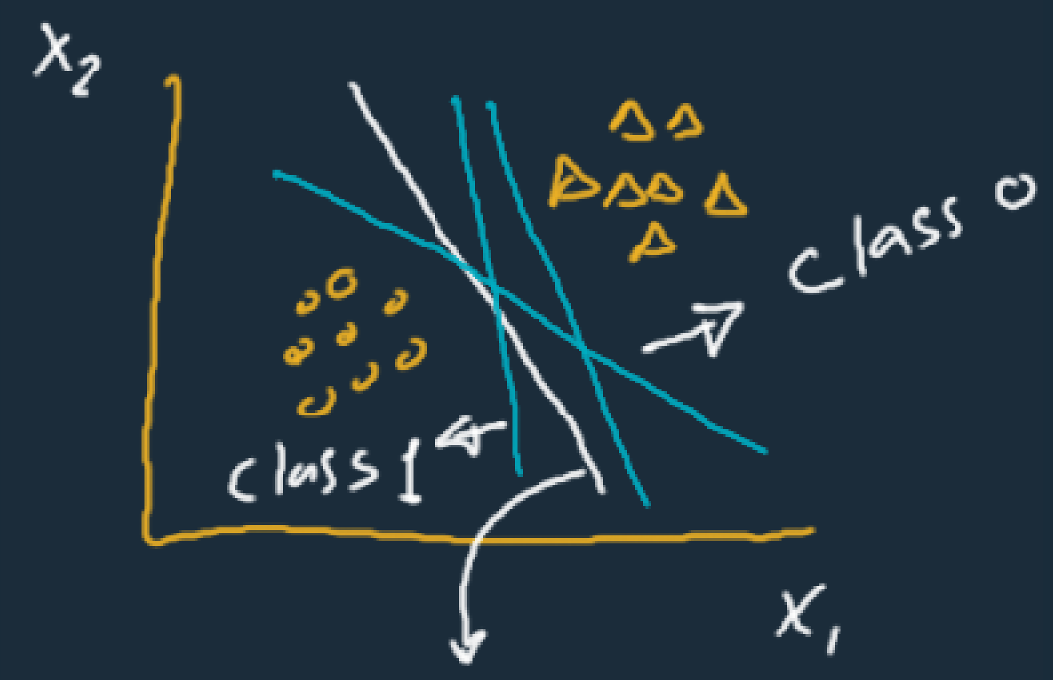
Linear SVC



Decision Boundary

$$\theta^T x = 0$$

- class 1 : $\theta^T x \geq +1$
- class 0 : $\theta^T x \leq -1$

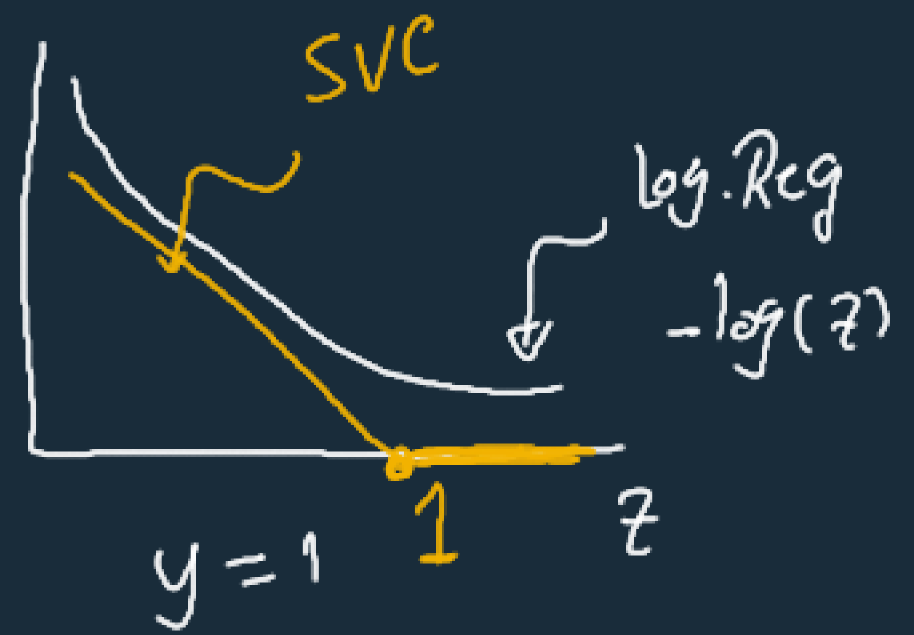


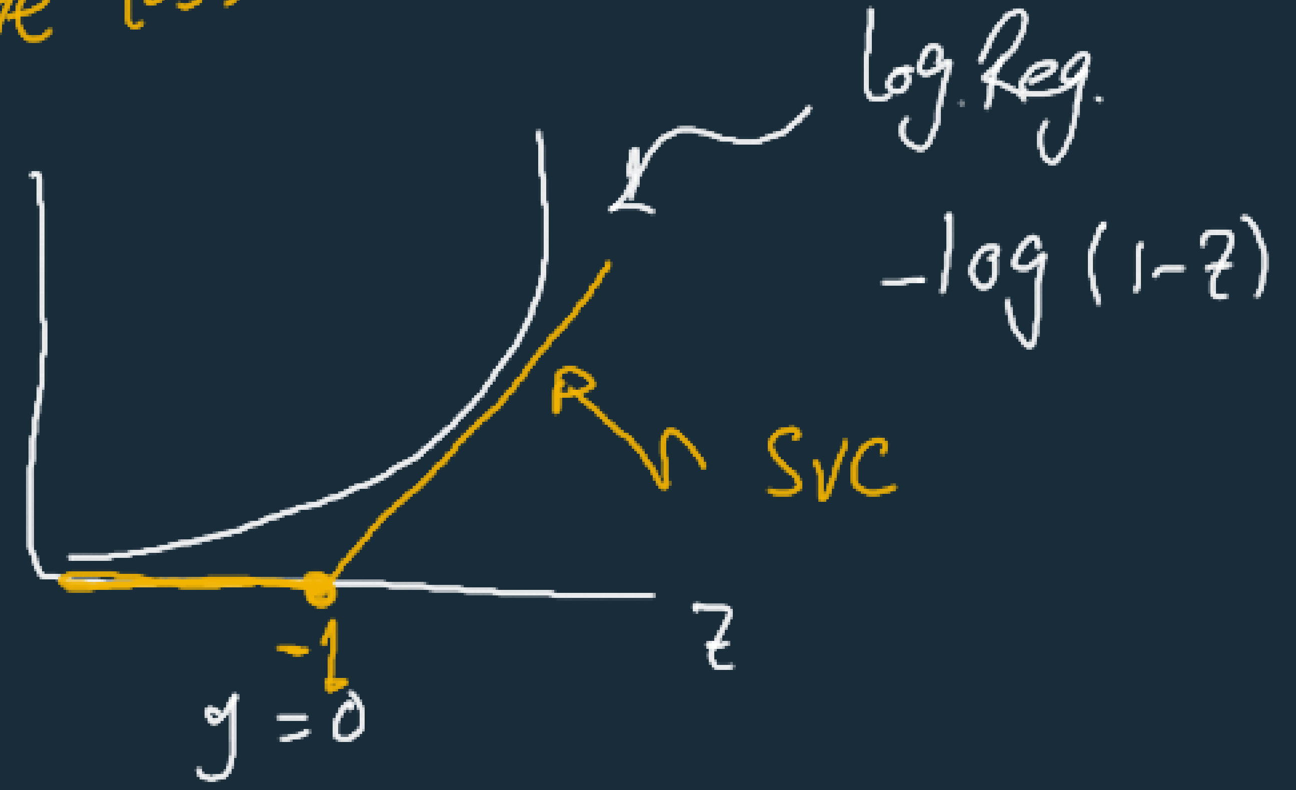
Best classifier

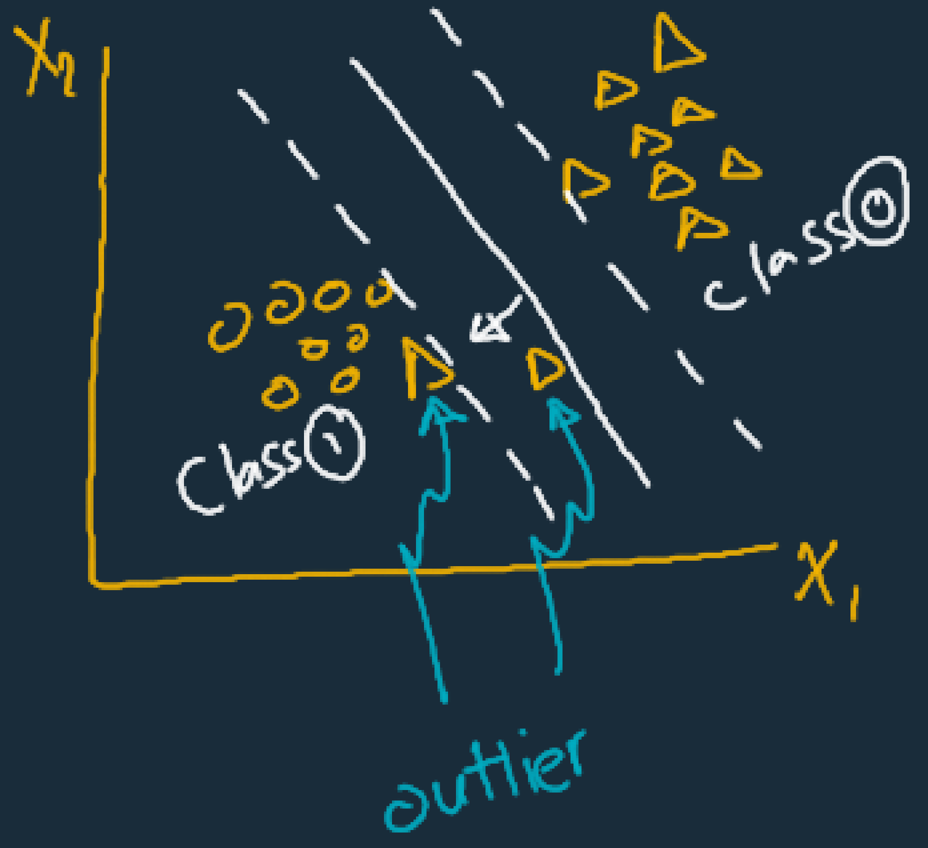
↳ Max. Margin

① Decision Boundary

② Cost f^n: hinge loss







① Hard Margin

↳ No Misclassification

② Soft Margin

↳ Allow Misclassification

SVM sinistive for scales

- feature Scaling

↳ Margin

- Regularization

↳ $C \propto 1/\text{Regularization}$

from Model

given

Boundary $\quad b + w_0 X_0 + w_1 X_1 = 0 \quad$ boundary $= -(b + w_0 X_0) / w_1$

Margin 1 $\quad b + w_0 X_0 + w_1 X_1 = +1 \quad$ margin1 $= \dfrac{-(b + w_0 X_0) + 1}{w_1}$

Margin 2 $\quad b + w_0 X_0 + w_1 X_1 = -1 \quad$ margin2 $= \dfrac{-(b + w_0 X_0) - 1}{w_1}$

X_0, boundary

X_0, margin1

X_0, margin2

Kernelized SVM

Kernel Trick → Getting the same result from "features Transformation" Without doing actual "Features Transformation"

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\,x_1 x_2 \\ x_2^2 \end{pmatrix}$$

(1) Actual Transformation

$$\phi(\mathbf{a})^\top\phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\,a_1 a_2 \\ a_2^2 \end{pmatrix}^\top \begin{pmatrix} b_1^2 \\ \sqrt{2}\,b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

Dot product

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\top \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}\right)^2 = (\mathbf{a}^\top\mathbf{b})^2$$

$$a = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \qquad b = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

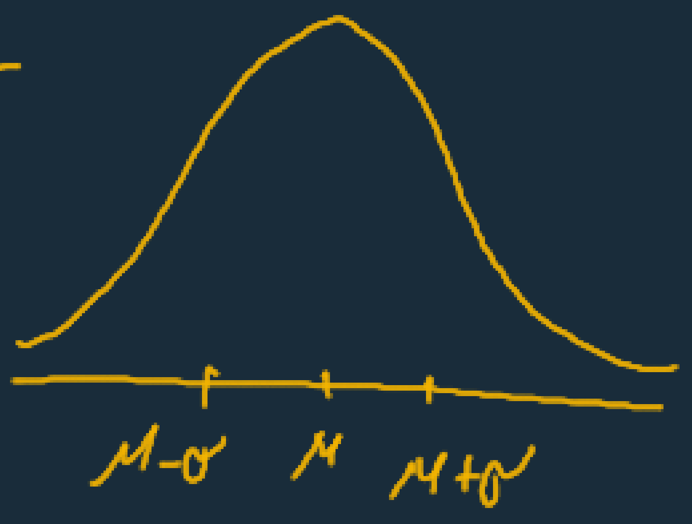
$$\phi(\mathbf{a})^\top\,\phi(\mathbf{b}) = (\mathbf{a}^\top\,\mathbf{b})^2$$

(2) Kernel Trick

Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$

Polynomial: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^\top \mathbf{b} + r)^d$

$$e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma \| \mathbf{a} - \mathbf{b} \|^2\right)$

x_1 landmark

l_1 l_2

$$\to x_2 = e^{-\gamma \| x_1 - l_1 \|^2}$$

=

$$\to x_3 = e^{-\gamma \| x_1 - l_2 \|^2}$$

=

SVC

- Linear
  - LinearSVC
  - SVC (Kernel = 'linear')

  ⇓

  n_features ↑

- Non linear
  - Transformation → linear SVC

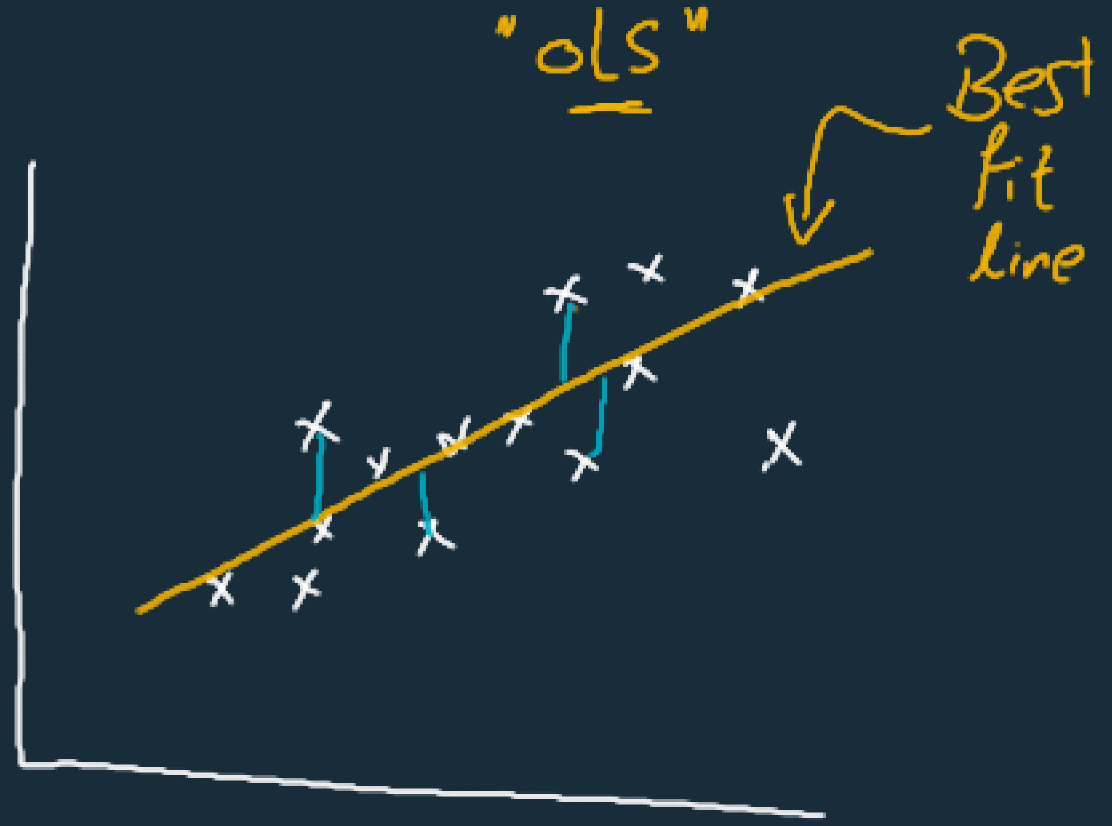
    impractical

    ↑ Computational Complexity

    - poly
    - rbf
  - SVC (Kernel → poly
    - rbf
    - ...)

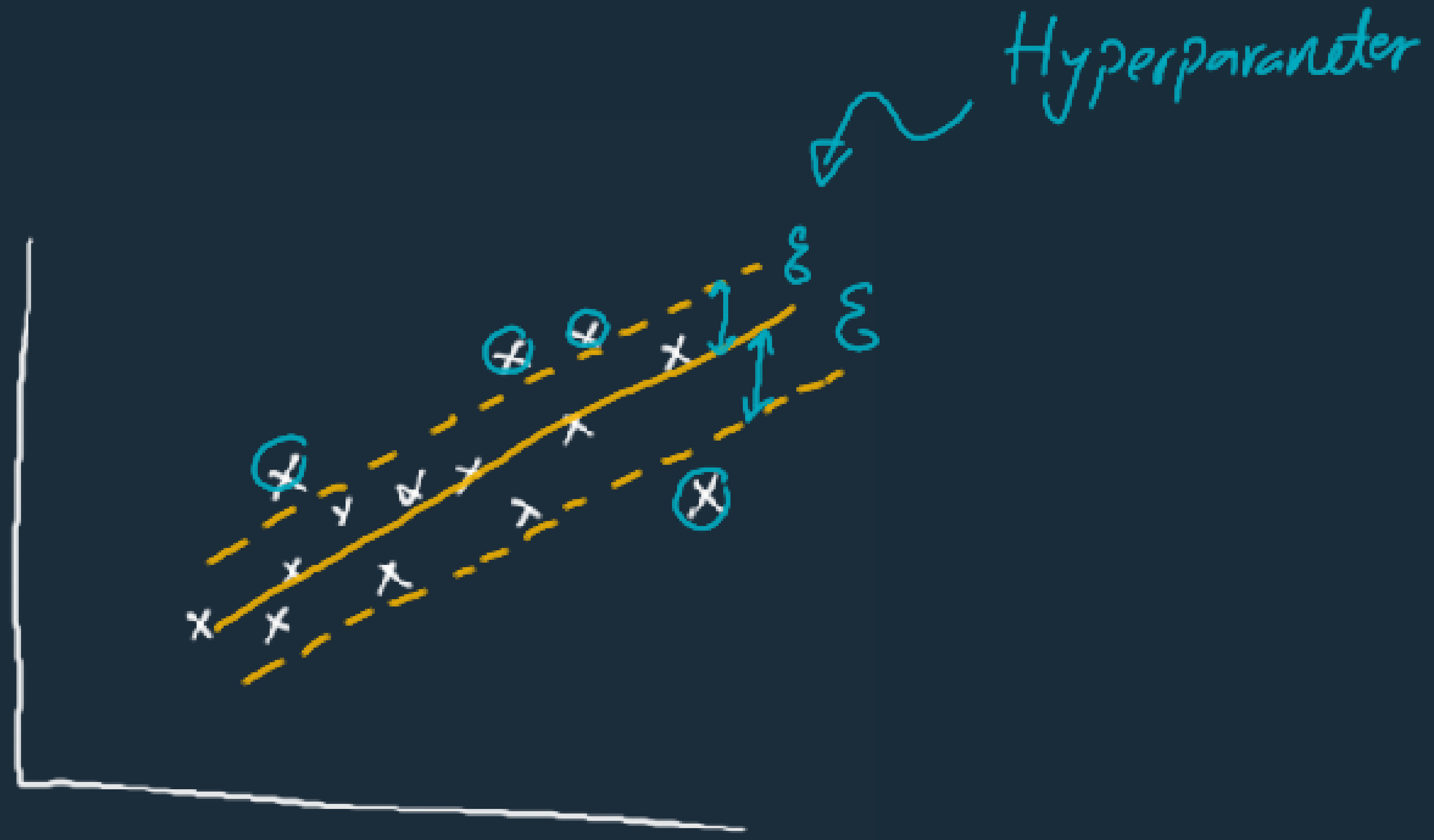
    get "feature Transf." Benefits without applying it actually

Support Vector Regressor



Linear Regression

Min MSE



Support Vector Regressor

Min instances out of the margin

SVR

"Kernel"

poly   rbf

Hyberparameters

Nonlinear → ε , degree, C , gamma